参考答案（书末集中）

先自己做，再看答案。对答案时顺序：① 先想什么 → ② 为什么这么写 → ③ 完整代码 → ④ 怎么验证。跳过前两步直接抄代码，等于白做。

阶段 1

【1-1】

① **先想什么**：`print` 可以接多个参数，分隔符由 `sep` 控制。

② **为什么这么写**：`sep` 默认是空格，题目要 `-`，就写 `sep="-"`；参数按名字、年龄、城市依次排。

③ **完整代码**：

```
print("小明", 19, "杭州", sep="-")
```

④ **怎么验证**：输出 小明-19-杭州；如果把 `sep` 删掉，输出变成 小明 19 杭州（空格）——对照即懂 `sep` 的作用。

【1-2】

① **先想什么**：只给一个参数 → 就是一个完整的字符串；一个参数根本用不到 `sep`。

② **为什么这么写**：字符串里可以包含空格，直接 `print("进度 50%")` 就满足要求。题面的"默认值"提示是干扰项：不写 `sep` 时用的就是默认值——但一个参数时它没有出场机会。

③ **完整代码**：

```
print("进度 50%")
```

④ **怎么验证**：输出 进度 50%。写成 `print("进度", "50%")` 是两个参数，虽然输出一样，但不满足"一个参数"。

【1-3】

① **先想什么**：变量 = 贴标签的盒子；重新赋值 = 把盒子里的东西换掉；`type` 看类型。

② **为什么这么写**：输出一次 → 改值 → 再输出，两次 `type` 都是 `int`（60 和 90 都是整数）。

③ **完整代码**：

```
score = 60
print(score)
score = 90
print(score)
print(type(score))
```

④ **怎么验证**：输出 60、90、<class 'int'>；若写成 `score = "90"`，`type` 会变成 <class 'str'>。

【1-4】

① **先想什么**：三个字面量的"长相"决定类型：不带小数点的是 int、带小数点的是 float、带引号的是 str。

② **为什么这么写**：Python 靠字面量形式自动判断类型，不需要声明。

③ **完整代码**：

```
print(type(5))
print(type(5.0))
print(type("5"))
```

④ **怎么验证**：三行输出分别是 <class 'int'>、<class 'float'>、<class 'str'>。一句话总结：数字的长相决定类型——无小数点是 int，有小数点是 float，有引号是 str。

【1-5】

① **先想什么**：分割线卡片 = 重复的 = 号 + 文字 + 重复的 = 号；长度用 `len`。

② **为什么这么写**：字符串重复用 `*`（`"=" * 4` 就是四个等号），拼接用 `+`。

③ **完整代码**：

```
line = "=" * 4 + "欢迎" + "=" * 4
print(line)
print("长度:", len(line))
```

④ **怎么验证**：输出 ====欢迎==== 和 长度: 10（4+2+4）；`len` 数的是字符个数。

【1-6】

① **先想什么**：f-string 把变量塞进字符串；控制小数位数用 `:.1f`。

② **为什么这么写**：`f"{变量:.1f}"` 保留一位小数；名字年龄城市各自放进 `{}`。

③ **完整代码**：

```
name = "小明"
age = 19
city = "杭州"
pi = 3.14159
print(f"{name}-{age}-{city}")
print(f"{pi:.1f}")
```

④ **怎么验证**：输出 小明-19-杭州、3.1；`:.1f` 是格式说明（四舍五入到一位）。

【1-7】

① **先想什么**：四个运算符各管一种运算；每行 print 后加注释说明结果含义。

② **为什么这么写**：`/` 真除法得 float、`//` 整除去尾、`%` 取余、`**` 乘方。

③ **完整代码**：

```
print(7 / 2) # 真除法：3.5
print(7 // 2) # 整除：3
print(7 % 2) # 取余：1
print(2 ** 10) # 2的10次方：1024
```

④ **怎么验证**：四个输出分别是 3.5、3、1、1024。

【1-8】

① **先想什么**：命名规则两条——不能数字开头；不能是关键字。

② **为什么这么写**：`1name` 数字开头，违反规则①；`if` 是关键字（语法词），被 Python 占用，违反规则②。

③ **完整代码**：本题文字作答：`1name` 以数字 1 开头，Python 变量名不能数字开头；`if` 是关键字，已经被语言本身占用了。

④ **怎么验证**：在交互式里输入 `1name = 1` 会报 SyntaxError；输入 `if = 1` 同样报 SyntaxError——用报错验证规则。

阶段 2

【2-1】

① **先想什么**：读入 → int 转数字 → if/elif/else 三分支。

② **为什么这么写**：`input` 返回字符串，比大小前必须 `int()`；三种情况正好 `elif` 链。

③ **完整代码**：

```
n = int(input("输入一个整数："))
if n > 0:
    print("正数")
elif n < 0:
    print("负数")
else:
    print("零")
```

④ **怎么验证**：分别输入 5、-3、0 测试三条分支；忘 `int()` 输入 5 会报 TypeError（字符串和 0 没法比）。

【2-2】

① **先想什么**：成绩分段 = elif 链，条件从高到低写。

② **为什么这么写**：从上往下依次判断，≥90 排第一，落到最后的 else 就是 <60。

③ **完整代码**：

```
score = int(input("输入成绩："))
if score >= 90:
    print("优秀")
elif score >= 75:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

④ **怎么验证**：输入 95/80/65/50 分别得四档；条件顺序反了（先 ≥60）会输出错误结果——这正是"顺序从高到低"的原因。

【2-3】

① **先想什么**：遍历 1~100，筛 7 的倍数；倍数 = 取余等于 0。

② **为什么这么写**：`range(1, 101)` 是 1 到 100（左闭右开）；`i % 7 == 0` 判断。

③ **完整代码**：

```
for i in range(1, 101):
    if i % 7 == 0:
        print(i)
```

④ **怎么验证**：输出 7、14、21 … 98，共 14 个。

【2-4】

① **先想什么**：编号遍历用 `enumerate`；序号从 1 开始用它的第二个参数。

② **为什么这么写**：`enumerate(names, 1)` 让 i 从 1 数而不是 0。

③ **完整代码**：

```
names = ["张三", "李四", "王五"]
for i, name in enumerate(names, 1):
    print(f"{i}号：{name}")
```

④ **怎么验证**：输出 1号：张三、2号：李四、3号：王五；不写 1 就变成 0 号开头。

【2-5】

① **先想什么**：while 循环里手动递减计数器；退出循环后再打印"发射！"。

② **为什么这么写**：while 比 for 多一个"自己管计数器"的动作，`n -= 1` 不能忘（忘了就死循环）。

③ **完整代码**：

```
n = 10
while n >= 1:
    print(n)
    n -= 1
print("发射！")
```

④ **怎么验证**：输出 10 到 1 再"发射！"；把 n -= 1 删掉会死循环（Ctrl+C 急救）。

【2-6】

① **先想什么**：随机数用 `random.randint(1, 100)`；猜数循环到猜对为止（不知道几次 → while）；计数变量每次 +1；对了 break。

② **为什么这么写**：`random` 是标准库要先 import；`while True` + `break` 实现"猜到为止"。

③ **完整代码**：

```
import random
target = random.randint(1, 100)
count = 0
while True:
    guess = int(input("猜一个1~100的数： "))
    count += 1
    if guess == target:
        print(f"猜对了！ 共猜了{count}次")
        break
    elif guess < target:
        print("小了")
    else:
        print("大了")
```

④ **怎么验证**：按提示缩小范围，猜中后打印次数；如果你自己改代码时去掉了 `int()`，str 和 int 比大小会报 TypeError——所以 `int()` 不能忘。

【2-7】

① **先想什么**：for = 知道次数（range 给你次数）；while = 不知道次数（靠条件）。

② **为什么这么写**：猜数字循环次数未知用 while；打印固定范围用 for。

③ **完整代码**：文字作答——知道循环次数用 `for`（例：打印 1~100 的 7 倍数）；不知道次数用 `while`（例：猜数字，猜到为止）。

④ **怎么验证**：口头能说清这两条要点即可。

【2-8】

① **先想什么**：外层 for 管行 i（1..9），内层 for 管列 j（1..i），打印 j×i=积，行尾换行。

② **为什么这么写**：第 i 行有 i 个式子；`print(..., end="\t")` 不换行且制表符对齐；内层循环完 `print()` 换行。

③ **完整代码**：

```
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{j}x{i}={i*j}", end="\t")
    print()
```

④ **怎么验证**：输出 9 行三角形乘法表，每行对齐；内层 range(1, i+1) 保证第 i 行有 i 个式子。

阶段 3

【3-1】

① **先想什么**：空列表 `[]`、`append` 加元素、`[-1]` 取最后一个。

② **为什么这么写**：`append` 追加到末尾，负数索引从尾部倒数。

③ **完整代码**：

```
nums = []
nums.append("苹果")
nums.append("香蕉")
nums.append("橙子")
print(nums[-1])
```

④ **怎么验证**：输出 橙子；`nums`[0] 是苹果、`nums`[2] 也是橙子。

【3-2】

① **先想什么**：切片 `[a:b]` 左闭右开。

② **为什么这么写**：前 3 个 = `[:3]`；后 2 个 = `[-2:]`（省略结尾 = 到末尾）；全部 = `[:]`。

③ **完整代码**：

```
lst = [0, 1, 2, 3, 4, 5]
print(lst[:3])
print(lst[-2:])
print(lst[:])
```

④ **怎么验证**：输出 [0, 1, 2]、[4, 5]、[0, 1, 2, 3, 4, 5]。

【3-3】

① **先想什么**：dict 增删改查 = `[]` 取值、`[]` 赋值改、**`del`** 删、print 看。

② **为什么这么写**：dict 用键访问；赋值给已有键 = 改、给新键 = 加；**`del`** 键 = 删。

③ **完整代码**：

```
d = {"hello": "你好"}
print(d["hello"])   # 查词
d["world"] = "世界"  # 加词
d["hello"] = "您好"  # 改词
del d["world"]         # 删词
print(d)
```

④ **怎么验证**：依次输出 你好、{'hello': '您好'}；用 [不存在的键] 会报 KeyError。

【3-4】

① **先想什么**：`get`(键, 默认值) 查不存在的键不报错；`items`() 遍历键值对。

② **为什么这么写**：`get` 比 `[]` 安全——查不到给默认提示而不是崩溃。

③ **完整代码**：

```
d = {"hello": "你好"}
print(d.get("goodbye", "查无此词"))
for k, v in d.items():
    print(k, "=", v)
```

④ **怎么验证**：输出 查无此词、hello = 你好。

【3-5】

① **先想什么**：set 去重 = 把列表塞进 `set`()；顺序会变因为 set 无序。

② **为什么这么写**：set 自动去重但元素没有先后。

③ **完整代码**：

```
names = ["小明", "小红", "小明", "小刚"]
s = set(names)
print(s)
```

④ **怎么验证**：输出类似 {'小红', '小明', '小刚'}（顺序每次可能不同）；`len`(s) = 3 说明重复的"小明"只留一个。顺序会变的原因：set 内部按哈希存放，不保留插入顺序。

【3-6】

① **先想什么**：存 3 个学生 = 列表装元组(姓名, 分数)；找最高分 = 逐个比，记录更大的。

② **为什么这么写**：元组适合"一组固定数据"；遍历时用下标 [1] 取分数比较。

③ **完整代码**：

```
students = [("小明", 92), ("小红", 88), ("小刚", 95)]
best = students[0]
for s in students:
    if s[1] > best[1]:
        best = s
print(f"最高分：{best[0]} {best[1]}分")
```

④ **怎么验证**：输出 最高分：小刚 95分；初始 best 取第一个，后面谁大换谁。

【3-7】

① **先想什么**：tuple 不可变、list 可变；数据固定不动用 tuple。

② **为什么这么写**：不可变 = 安全，防止误改。

③ **完整代码**：文字作答——固定不变的数据用 tuple（例：一个点的经纬度 (120.2, 30.3)，天生一对固定坐标，不该被修改）；会增删改的数据用 list（例：通讯录名单）。

④ **怎么验证**：试试 `t = (1, 2)` 然后 `t[0] = 5` 会报 TypeError——用报错验证"不可变"。

【3-8】

① **先想什么**：循环菜单 = `while True` + `input` + `if` 分支；1 添加（dict 赋值）、2 查询（get）、3 退出（break）。

② **为什么这么写**：dict 存姓名→电话；`get` 查询给默认提示。

③ **完整代码**：

```
contacts = {}
while True:
    cmd = input("1添加 2查询 3退出：")
    if cmd == "1":
        name = input("姓名：")
        phone = input("电话：")
        contacts[name] = phone
        print("已添加")
    elif cmd == "2":
        print(contacts.get(input("查谁："), "查无此人"))
    elif cmd == "3":
        print("再见")
        break
```

④ **怎么验证**：完整走一遍添加→查询→退出；查询不存在的人输出"查无此人"。

【3-9】

① **先想什么**：一行日志四步加工——去空白 → 切开 → 取第 3 块（级别）并统一大小写 → 判断开头。四步正好用四个字符串方法。

② **为什么这么写**：`strip()` 去首尾空白（读文件/输入必用）；`split()` 按空格切开成列表；`lower()` 统一小写（"ERROR" 和 "error" 都当 error）；`startswith()` 判断开头——四个方法各用一次，一条流水线。

③ **完整代码**：

```
line = input("输入一行日志：")
parts = line.strip().split()
level = parts[2].lower()
print(level.startswith("err"))
```

④ **怎么验证做对了**：三种输入各跑一遍——① `2026-08-25 14:30:12 ERROR 磁盘满`（首尾有空格）→ True；② `2026-08-25 14:30:12 error 磁盘满`（小写）→ True；③ `2026-08-25 14:30:12 INFO 正常` → False。三个结果都对，说明 strip/split/lower/startswith 四件套都懂了。

阶段 4

【4-1】

① **先想什么**：def 定义函数，函数名 + 括号调用；调用 3 次就写 3 行。

② **为什么这么写**：def 只是"登记"，调用才执行；连调 3 次 = 复用 3 次。

③ **完整代码**：

```
def hello():
    print("我是新函数")
hello()
hello()
hello()
```

④ **怎么验证**：输出 3 行"我是新函数"；只 def 不调用则一个字都不输出。

【4-2】

① **先想什么**：默认参数 = 定义时给参数预先赋值；不传参用默认，传参就覆盖。

② **为什么这么写**：greeting 带默认值"你好"；调用时省略 = 用默认，写第二个参数 = 自定义。

③ **完整代码**：

```
def greet(name, greeting="你好"):
    print(f"{greeting}，{name}")
greet("小明")
greet("小明", "早上好")
```

④ **怎么验证**：输出 你好，小明 和 早上好，小明。

【4-3】

① **先想什么**：两个数比大小，用 return 把两个结果一起交回（多返回值）。

② **为什么这么写**：return 后面逗号分隔多个值，实际返回一个元组；接收时用两个变量拆包。

③ **完整代码**：

```
def min_max(a, b):
    if a > b:
        return b, a
    return a, b
small, big = min_max(7, 3)
print(small, big)
```

④ **怎么验证**：输出 3 7；换 min_max(2, 9) 输出 2 9。

【4-4】

① **先想什么**：函数里给变量赋值，默认新建**局部变量**，不碰外面的同名变量。

② **为什么这么写**：函数有自己的小房间（作用域）；函数里 x = 99 是房间里的新标签，房间外的 x 还是 10。

③ **完整代码**：

```
x = 10
def change():
    x = 99
    print("函数内:", x)
change()
print("函数外:", x)
```

④ **怎么验证**：输出 函数内: 99、函数外: 10——外面的 x 没变，证明函数内赋值不会改动外部同名变量。

【4-5】

① **先想什么**：当前目录用 os.getcwd()，列目录用 os.listdir()；用之前先 import os。

② **为什么这么写**：os 是标准库自带；getcwd = get current working directory（取当前工作目录），listdir = list directory（列目录）。

③ **完整代码**：

```
import os
print(os.getcwd())
print(os.listdir())
```

④ **怎么验证**：先输出当前目录路径，再输出该目录下所有文件/文件夹的名字列表。

【4-6】

① **先想什么**：两种导入写法——from ... import 直接拿函数、import ... as 起别名。

② **为什么这么写**：from os import getcwd 后直接 getcwd()；import time as t 后 t.time()。别名可以缩短长模块名。

③ **完整代码**：

```
from os import getcwd
import time as t
print(getcwd())
print(t.time())
```

④ **怎么验证**：输出当前目录和一串时间戳（从 1970-01-01 起的秒数）。

【4-7】

① **先想什么**：把"添加联系人"这段代码搬进 def 函数，主体菜单里调它。

② **为什么这么写**：重复/独立的一段逻辑收进函数，主体更清楚；以后要改添加逻辑只改一处。

③ **完整代码**：

```
contacts = {}
def add_contact():
    name = input("姓名：")
    phone = input("电话：")
    contacts[name] = phone
    print("已添加")
# 主体：调用函数
add_contact()
print(contacts)
```

④ **怎么验证**：运行后走一遍添加流程，print(contacts) 能看到刚加的人。

【4-8】

① **先想什么**：return 把结果**交回来**（能存变量）；print 只是**显示**（返回值是 None）。

② **为什么这么写**：两个函数做同一件事但一个 return 一个 print，用变量接收对比结果。

③ **完整代码**：

```
def with_return(x):
    return x * 2
def with_print(x):
    print(x * 2)
a = with_return(3)
b = with_print(3)
print("a =", a)
print("b =", b)
```

④ **怎么验证**：输出顺序是 6（with_print 里打印的）、a = 6、b = None。总结：return 的结果能接着用（a=6）；print 只是给人看的，函数返回值是 None（b=None）。

阶段 5

【5-1】

① **先想什么**：整段读用 read()；逐行读用 for 循环遍历文件对象。

② **为什么这么写**：with open 自动关文件；for line in f 每次读一行；strip() 去掉行尾换行。

③ **完整代码**：

先在**终端**执行建文件（不是 Python 代码）：`mkdir -p ~/lab; echo '第一行' > ~/lab/test.txt; echo '第二行' >> ~/lab/test.txt`；再 `cd ~/lab` 跑下面的 Python（本阶段所有文件都放这里，相对路径才找得到）。

```
with open("test.txt", "r", encoding="utf-8") as f:
    print(f.read())
with open("test.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

④ **怎么验证**：整段打印一次、逐行打印一次；不 strip 的话逐行输出间会多空行（行尾换行 + print 自带换行）。

【5-2】

① 先想什么："w" 打开时清空原文件，"a" 打开时接着末尾写。

② 为什么这么写：先 w 写第一次，再 a 写第二次，最后读出来看——结果只有两行，且第二行是 a 追加的。

③ 完整代码：

```
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("第一次\n")
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("第二次\n")
with open("log.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

④ 怎么验证：输出 第一次 和 第二次 两行。如果第二次也用 w，最终只剩"第二次"——这就是覆盖与追加的区别。

【5-3】

① 先想什么：dict 存成 JSON 用 json.dump；读回用 json.load；先 import json。

② 为什么这么写：json.dump(对象, 文件) 写入；ensure_ascii=False 让中文正常显示不转码；读回后 load 得到原样的 dict。

③ 完整代码：

```
import json
d = {"name": "小明", "age": 19}
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(d, f, ensure_ascii=False)
with open("data.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded)
```

④ 怎么验证：输出 {'name': '小明', 'age': 19}；用 cat 看 data.json 应是一行 JSON 文本。

【5-4】

① **先想什么**：启动时若 JSON 文件存在就读回通讯录——用 os.path.exists（判断文件是否存在，os 模块的常用函数）检查，避免首次运行读不存在的文件报错；退出时把 dict 存进 JSON。

② **为什么这么写**：os.path.exists 判断文件在不在，避免首次运行读不存在文件报错；"3"分支里 dump 保存。

③ **完整代码**：

```
import json, os
FILE = "contacts.json"
contacts = {}
if os.path.exists(FILE):
    with open(FILE, "r", encoding="utf-8") as f:
        contacts = json.load(f)
while True:
    cmd = input("1添加 2查询 3保存退出：")
    if cmd == "1":
        name = input("姓名：")
        phone = input("电话：")
        contacts[name] = phone
    elif cmd == "2":
        print(contacts.get(input("查谁："), "查无此人"))
    elif cmd == "3":
        with open(FILE, "w", encoding="utf-8") as f:
            json.dump(contacts, f, ensure_ascii=False)
        print("已保存")
        break
```

④ **怎么验证**：添加一个人 → 选 3 退出 → 再运行程序 → 选 2 能查到刚添加的人（数据跨运行存活）。

【5-5】

① **先想什么**：读文件用 open()；文件不存在时它会抛出 FileNotFoundError。要接住这个错误，就用 **try**（尝试）包住读文件的代码，用 **except FileNotFoundError**（接住"文件不存在"）处理报错分支。

② **为什么这么写**：open() 在文件不存在时必然报错，与其让程序崩溃，不如主动接住并给友好提示。指定异常类型 FileNotFoundError 而不是裸 **except**，是为了只处理"文件不存在"这一种情况，其他错误仍应暴露出来。

③ **完整代码**：

```
try:
    with open("不存在.txt", "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("这个文件不存在，请检查路径")
```

④ **怎么验证**：跑一遍应打印友好提示（而不是红色 Traceback）；把文件名改成一个真实存在的文件再跑，正常打印内容——两种路径都对，说明 try/except 正确。

【5-6】

① **先想什么**：在 5-5 基础上加 finally——无论成功失败都执行的收尾代码。

② **为什么这么写**：finally 天生是"善后"位置（关文件、发通知），出错不出错都跑。

③ **完整代码**：

```
try:
    with open("不存在.txt", "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("这个文件不存在，请检查路径")
finally:
    print("本次读取结束")
```

④ **怎么验证**：文件不存在时输出两行（提示 + 本次读取结束）；换成存在的文件也输出"本次读取结束"——两种走向 finally 都执行。

【5-7】

① **先想什么**：行数 = 列表长度 len(lines)；字数 = 每行去掉末尾换行后数一数（len(line.strip())）累加。

② **为什么这么写**：readlines 得到行列表；len(line.strip()) 数一行的字符；循环累加出总字数。

③ **完整代码**：

```
with open("test.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
total = 0
for line in lines:
    total += len(line.strip())
print(f"共{len(lines)}行，{total}个字")
```

④ **怎么验证**：对照文件人工数一遍；报告里的数字与文件一致。

【5-8】

① **先想什么**："w" = 清空重写、"a" = 末尾追加；对应 Linux 的重定向符号。

② **为什么这么写**：两个模式分别对应 Linux 的覆盖/追加语义，联想记忆。

③ **完整代码**：文字作答——"w"（覆盖）打开时先把文件清空再写，对应 Linux 的 >；"a"（追加）从末尾接着写，对应 Linux 的 >>。

④ **怎么验证**：口头说清 + 用 5-2 的实验现象佐证。

阶段 6

【6-1】

① **先想什么**：requests.get 请求 URL；r.status_code 看状态码；r.json() 解析 JSON。

② **为什么这么写**：Open-Meteo 是免注册的公开天气 API；timeout 防止卡死；json() 把响应文本转成 dict。

③ **完整代码**：

```
import requests
r = requests.get("https://api.open-meteo.com/v1/forecast?latitude=30.25&longitude=120.17&current_weather=true", timeout=10)
print(r.status_code)
print(r.json())
```

④ **怎么验证**：输出 200 和一长串 JSON（含 current_weather）；URL 改错会输出 400/404。

【6-2】

① **先想什么**：连不上的错都属于 RequestException 一族，用 try/except 接住它输出友好提示。

② **为什么这么写**：RequestException 是 requests 所有网络异常的基类，接住它 = 接住"连不上"；用**本机未监听的 127.0.0.1:9**——连接必然被拒绝，比域名更确定（域名类失败在代理环境下可能被拦成 200 造成无输出）。（若按题面用"不存在的主机"，走 DNS 失败分支，同样会被 RequestException 接住。）

③ **完整代码**：

```
import requests
try:
    requests.get("http://127.0.0.1:9", timeout=3)
except requests.exceptions.RequestException:
    print("网络不通")
```

④ **怎么验证**：运行输出"网络不通"且不报 Traceback；把 URL 换成真实网站（如 6-1 用过的接口）则正常返回（无输出即成功）。若你的环境有透明代理把 127.0.0.1:9 也拦成响应，说明网络层被改写，属环境差异——此时在题面注明实际结果即可。

【6-3】

① **先想什么**：argparse 三步——建解析器、加参数、解析；用 args.name 取值。

② **为什么这么写**：--name 定义命令行参数；required=True 强制必传；args.name 拿值。

③ **完整代码**：

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--name", required=True)
args = p.parse_args()
print(f"你好，{args.name}")
```

④ **怎么验证**：python3 脚本.py --name 小明 输出"你好，小明"；不带参数运行会被拦截并提示缺少 --name。

【6-4】

① **先想什么**：default 给默认值；help 写中文说明（--help 时显示）。

② **为什么这么写**：default="世界" 让参数可选；help 让工具自带说明书。

③ **完整代码**：

```
import argparse
p = argparse.ArgumentParser(description="打招呼小工具")
p.add_argument("--name", default="世界", help="你的名字")
args = p.parse_args()
print(f"你好，{args.name}")
```

④ **怎么验证**：不带参数输出"你好，世界"；--name 小明 输出"你好，小明"；--help 显示 description 和"你的名字"。

【6-5】

① **先想什么**：logging.basicConfig 配日志到文件；info 记正常、error 记异常。

② **为什么这么写**：basicConfig 一次性配好文件名、级别、格式；故意制造 ZeroDivisionError 让 except 分支记一条 error。

③ **完整代码**：

```
import logging
logging.basicConfig(filename="app.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
logging.info("开始")
try:
    1 / 0
except ZeroDivisionError as e:
    logging.error("出错了：%s", e)
logging.info("结束")
```

④ **怎么验证**：运行无打印；**先 rm -f app.log 再运行**（logging 默认追加，旧日志行不会消失——每次验证前清掉或换新文件名）。cat app.log 应看到**三条本次新增**，且顺序对应代码顺序：① INFO 开始 ② ERROR 出错了：division by zero ③ INFO 结束。level=INFO 时 INFO 和 ERROR 都能进文件，缺哪条查哪步。

【6-6】

① **先想什么**：改 basicConfig 的 level 为 WARNING，再跑 6-5 的脚本观察 INFO 行消失。

② **为什么这么写**：级别过滤机制——只记录"级别 ≥ 设定值"的日志；WARNING 比 INFO 高，INFO 被过滤。

③ **完整代码**：把 6-5 的 `level=logging.INFO` 改成 `level=logging.WARNING`，其余不变，重跑。

④ **怎么验证**：同样**先 rm -f app.log 清旧日志**再重跑（否则会混着上次的行）。cat app.log 只剩**一条 ERROR**：INFO 的"开始/结束"都被过滤了——WARNING 级别高于 INFO，INFO 没资格进文件。级别排序：DEBUG < INFO < WARNING < ERROR < CRITICAL。

【6-7】

① **先想什么**：subprocess.run 执行系统命令；capture_output 捕获、text 转文本。

② **为什么这么写**：命令写成列表 ["df", "-h"]；stdout 里就是命令输出。

③ **完整代码**：

```
import subprocess
r1 = subprocess.run(["df", "-h"], capture_output=True, text=True)
r2 = subprocess.run(["free", "-h"], capture_output=True, text=True)
print(r1.stdout)
print(r2.stdout)
```

④ **怎么验证**：输出和终端里跑 df -h、free -h 一模一样。

【6-8】

① **先想什么**：三个知识点组合——argparse 收参数、subprocess 跑命令、logging 记日志。流程：解析参数 → 按参数选命令 → 执行并打印 → 记一条日志。

② **为什么这么写**：参数用 choices 限定只接受 disk/mem，传错自动报错；capture_output=**True** + text=**True** 捕获输出；日志用 logging.basicConfig 配到文件。

③ **完整代码**：

```
import argparse
import logging
import subprocess

logging.basicConfig(filename="check.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

parser = argparse.ArgumentParser()
parser.add_argument("--check", choices=["disk", "mem"], required=True, help="查什么：disk 或 mem")
args = parser.parse_args()

cmd = ["df", "-h"] if args.check == "disk" else ["free", "-h"]
r = subprocess.run(cmd, capture_output=True, text=True)
print(r.stdout)
logging.info("检查了 %s", args.check)
```

④ **怎么验证**：--check disk 显示磁盘情况、--check mem 显示内存情况；--check cpu 报"无效选择"；check.log 里每次运行多一条 INFO。四种行为都对即通过。

附录：与学习路线的对应关系

练习册位置	路线对应	达成标志
阶段 5	检查点②	能独立写带 try/except 的脚本
阶段 6	补充模块（requests/argparse/logging/subprocess）	模块全部落地为可运行代码
进阶篇场景 1	脚本 1：批量文件重命名	产出脚本 +1
进阶篇场景 2	脚本 2：天气查询器	产出脚本 +1
进阶篇场景 3	王牌项目 server-health-check 版本 0	王牌项目正式起步
进阶篇场景 4	脚本 3：Nginx 日志分析器	产出脚本 +1
进阶篇场景 5	版本 2 paramiko 远程版（选做）	远程巡检能力

卷末速查 · 这册用到的模块与词

模块	用到的词	一句话
os / os.path	listdir · exists	列目录、查在不在
json	load · dump	读写 JSON 文件
random	randint	随机整数
time	time()（配合 import time as t 的写法示例）	取当前时间戳
requests	get	发网络请求（第 6 阶段）
argparse / logging / subprocess	ArgumentParser / basicConfig / run	参数、日志、调命令（第 6 阶段）
内置	open/with · len · enumerate · append · get · items · split · strip	开文件、计数、编号循环、容器操作、字符串处理

相关词（**本册正文里没用到**，看到别懵——见《06 进阶篇》与《12 内置函数手册》）：`sorted` · `replace` · `os.path.join` · `os.path.isfile` · `time.strftime`。